

Logical Planning Laboratory

Advay Matapurkar | ID: 2023A7PS0459G

CS F407 Artificial Intelligence – Week 4: Logic + Search = Planning

Files: `planner.py` (planner and tests), `run_output.txt` (full run). Optional Task 5 and the optional Prolog extension (Tasks 6–8) were not attempted.

1. Specification of the planning problem (Task 0)

(a) **Initial state** $I = \{At(Robot,A), At(Package,A)\}$.

(b) **Goal** $G = \{At(Package,C)\}$.

Locations A, B, C. Connections: $A \leftrightarrow B$ and $B \leftrightarrow C$ (no direct A–C link).

States are sets of propositions; anything not in the set is false (closed-world assumption).

(c, d) **Actions, preconditions and effects:**

Action	Preconditions	Effects
Move(x,y), for (x,y) in $\{(A,B),(B,A),(B,C),(C,B)\}$	$At(Robot,x)$	$\neg At(Robot,x), At(Robot,y)$
PickUp(Package,l), l in {A,B,C}	$At(Robot,l), At(Package,l)$, and (added during review) $\neg Holding(Package)$	$\neg At(Package,l), Holding(Package)$
Drop(Package,l), l in {A,B,C}	$At(Robot,l), Holding(Package)$	$\neg Holding(Package), At(Package,l)$

Applicability in I. An action is applicable in S only if $S \models \text{Preconditions}(a)$, i.e. every precondition holds; being in the action list is not enough.

- **PickUp(Package,A) is applicable.** Its preconditions $At(Robot,A)$ and $At(Package,A)$ are both in I (and $Holding(Package)$ is absent, so the negative precondition holds).

- **Drop(Package,C) is not applicable.** It needs $At(Robot,C)$, which is false (the robot is at A), and $Holding(Package)$, which is also false. Either failure alone is enough.

The program confirms both results (first lines of the run output).

2. Manually constructed plan (Task 1)

The robot is already at the package, so it can pick it up at A and carry it (the handout's example moves to B first, which is also valid but longer):

State	Action	Facts
S0	–	$At(Robot,A), At(Package,A)$
S1	PickUp(Package,A)	$At(Robot,A), Holding(Package)$
S2	Move(A,B)	$At(Robot,B), Holding(Package)$
S3	Move(B,C)	$At(Robot,C), Holding(Package)$
S4	Drop(Package,C)	$At(Robot,C), At(Package,C) \Rightarrow S4 \models G$

Each step's preconditions hold in the previous state (e.g. $Move(B,C)$ needs $At(Robot,B)$, true in S2). Four actions is the minimum: the package must be picked up, the graph forces two moves $A \rightarrow B \rightarrow C$, and it must be dropped. The BFS planner returned exactly this plan.

3. Prompt used with the LLM (Task 2)

The LLM was Claude. I wrote the prompt below: it is based on the suggested prompt in the handout, extended with the warehouse problem and its actions as specified in section 1. It is reproduced verbatim.

I want to implement a simple planning agent in Python.

Represent a state as a set of logical propositions (strings such as `"At(Robot,A)"`).

Each action should contain:

- a name;

- positive preconditions;
- negative preconditions;
- positive effects;
- negative effects.

An action is applicable if all of its preconditions are satisfied by the current state (all positive preconditions are in the state, no negative precondition is in the state).

When an action is applied:

1. remove its negative effects from the state;
2. add its positive effects to the state.

Use breadth-first search to find a sequence of actions that achieves a specified goal.

The program should also:

- detect when no plan exists and print "No plan found";
- print the resulting sequence of actions;
- print the states reached after each action.

The problem to solve is a warehouse robot.

Locations: A, B, C. The robot can move only along these connections: A<->B and B<->C (there is no direct A-C connection).

Initial state: {At(Robot,A), At(Package,A)}

Goal: {At(Package,C)}

Actions:

- Move(x,y) for connected x,y (A->B, B->A, B->C, C->B):
precondition At(Robot,x); effects: remove At(Robot,x), add At(Robot,y).
- Pickup(Package,l) for each location l:
preconditions At(Robot,l) and At(Package,l);
effects: remove At(Package,l), add Holding(Package).
- Drop(Package,l) for each location l:
preconditions At(Robot,l) and Holding(Package);
effects: remove Holding(Package), add At(Package,l).

Explain the implementation and identify any assumptions you make.

Run the generated program on the warehouse problem.

Where each specification idea appears in the code: *Preconditions* → `applicable()` (positive preconditions are a subset of the state, negative preconditions are disjoint from it). *Effects* → `apply_action()` (state minus negative effects, union positive effects). *Goal* → the test `goal <= state` in `bfs_plan()`, which terminates the search. *BFS* → the FIFO deque frontier plus a visited set, so shorter plans are explored before longer ones and no state is expanded twice.

Attribution of code

Part	Source
Action dataclass, <code>applicable()</code> , <code>apply_action()</code> , <code>bfs_plan()</code> , warehouse action definitions	Generated by the LLM from the prompt above.
<code>validate_plan()</code> – independent replay that checks each precondition and the goal	Added/modified during review: it deliberately shares no code with <code>bfs_plan()</code> .
Negative precondition <code>¬Holding(Package)</code> on <code>PickUp</code>	Modified during review, so the planner has a negative precondition. In the warehouse problem it never decides applicability by itself, so a separate test exercises it (section 5).
Tests A, B, C1–C3, <code>invalid-plan</code> , <code>negative-precondition</code> and <code>trivial-goal</code> checks; <code>report()</code>	Written with LLM assistance; test design reviewed against the handout.

4. The generated Python program

The program is submitted as a separate file, `planner.py`. Its provenance is listed in the attribution table above.

5. Test results (Task 3)

Test	Initial	Goal	Plan found?	Plan	Actually valid?
A: original problem	Robot@A, Pkg@A	At(Package, C)	Yes	PickUp(A), Move(A,B), Move(B,C), Drop(C)	Yes – validator checked all 4 steps
B: PickUp removed	Robot@A, Pkg@A	At(Package, C)	No	"No plan found"	n/a – correctly reported impossible

Test	Initial	Goal	Plan found?	Plan	Actually valid?
C1: only Move actions	Robot@A, Pkg@A	At(Package, C)	No	"No plan found"	n/a – robot reaching C is not package reaching C
C2: only Move actions	Robot@A, Pkg@A	At(Robot,C)	Yes	Move(A,B), Move(B,C)	Yes – package still at A, as it should be
C3: full actions + robot-only Teleport(A,C)	Robot@A, Pkg@A	At(Package, C)	Yes	PickUp(A), Teleport(A,C), Drop(C)	Yes – shorter (3 steps) but the package is still carried

Interpretation. Test B shows the planner reports failure instead of inventing an action. Tests C1/C2 show the planner tracks the package and robot as separate propositions: reaching C with the robot never satisfies $At(Package,C)$. In C3 the extra robot-only action shortens the plan, but the planner still needed $PickUp$ and $Drop$ to satisfy the goal, so it did not produce a bogus plan. Extra checks: the plausible-looking plan $Move(A,B), Move(B,C), Drop(C)$ is rejected by the validator because $Holding(Package)$ was never established; in an artificial state $\{At(Robot,A), At(Package,A), Holding(Package)\}$ the positive preconditions of $PickUp(Package,A)$ hold but the action is not applicable, and the validator rejects it, because the negative precondition $\neg Holding(Package)$ fails; and a goal already true in the initial state yields the empty plan. The assertions in `planner.py` all passed ("All tests passed").

Full console output is in `run_output.txt`. Excerpt (Test A):

```
Initial: {At(Package,A), At(Robot,A)}
Goal:    {At(Package,C)}
Result: plan of length 4: PickUp(Package,A) -> Move(A,B) -> Move(B,C) -> Drop(Package,C)
S0: {At(Package,A), At(Robot,A)}
S1 after PickUp(Package,A): {At(Robot,A), Holding(Package)}
S2 after Move(A,B): {At(Robot,B), Holding(Package)}
S3 after Move(B,C): {At(Robot,C), Holding(Package)}
S4 after Drop(Package,C): {At(Package,C), At(Robot,C)}
[validator] step 1 PickUp(Package,A): preconditions OK
[validator] step 2 Move(A,B): preconditions OK
[validator] step 3 Move(B,C): preconditions OK
[validator] step 4 Drop(Package,C): preconditions OK
[validator] goal satisfied
Plan valid: True
```

6. Answers to “Think About It” and Task 4

Task 0: applicability.

An action is not applicable just because it is listed; it is applicable in S only when every precondition is satisfied by S . This is the logical-reasoning step: checking $S \models \text{Preconditions}(a)$.

Task 2: mapping the specification to code.

See section 3: preconditions $\rightarrow$ `applicable`, effects $\rightarrow$ `apply_action`, goal $\rightarrow$ termination test in `bfs_plan`, BFS $\rightarrow$ the queue of frontier states.

Task 4: completing the pipeline.

```
Current state
|
Check action preconditions      (logic: S |= Preconditions(a) ?)
|
Apply the action's effects      (logic: S' = (S - NegEffects) U PosEffects)  <-- the "?" step
|
Generate successor state
|
Search over alternatives        (BFS queue, visited set)
|
Goal? (G subset of S ?)  yes: return plan | no: expand next state | queue empty: No plan found
```

How logic and search work together. Logic decides what is possible: for a state S it tells us which actions are applicable and exactly what the successor state looks like. Search decides what to try: it treats states as nodes and applicable actions as edges, and BFS systematically explores them in order of plan length, avoiding repeats, until a state entails the goal. Without the logical check, search would generate impossible

moves; without search, the logic would have no way of choosing among the many applicable actions. Logic determines what is possible; search determines what to try.

7. Reflection questions

1. Why specify preconditions and effects before asking an LLM to write the planner?

They are the definition of the problem. With a precise specification the LLM's output can be checked against it; a vague request ("solve the warehouse problem") lets the model make up its own semantics, which I could not then test.

2. Give an example of an error that could occur if preconditions were not checked.

The robot could Drop(Package,C) at the start although it is at A and not holding anything, making At(Package,C) true from nowhere. The planner would return a one-step "plan" that no real robot could execute (my validator rejects the analogous plan).

3. Why is a plan that looks reasonable not necessarily valid?

Validity is a property of the state transitions, not of how the plan reads. Move(A,B), Move(B,C), Drop(C) looks like delivery but the package was never picked up, so Drop's precondition Holding(Package) fails.

4. What did the LLM contribute?

The boilerplate engineering: the action representation, applicability and application functions, the BFS loop with a visited set, the warehouse action definitions, and the printing of plans and states.

5. What did I have to verify independently?

That preconditions are checked before every expansion, that negative effects are removed before positive ones are added, that the goal test is a subset test, that impossible problems return "No plan found", and that returned plans are valid. I did this with the separate validate_plan() replay and by hand-computing the expected plan in Task 1.

6. Where is logical reasoning used?

In deciding applicability ($S \models \text{Preconditions}(a)$), in computing successor states from effects, and in checking that the final state entails the goal.

7. How is planning related to the search algorithms from the previous module?

Planning is search in a state space: states are nodes, applicable actions are edges, and the goal is a set of goal states. The planner here is uninformed breadth-first search; it is complete and returns a shortest plan because every action costs 1. A heuristic such as the number of unmet goal propositions could turn it into A*.

Takeaway

Understand → Specify → Generate → Execute → Verify. The LLM helped with the implementation, but the agent is trustworthy only because it was tested and its plans were checked by independent code.